Types of Asymptotic Notations in Complexity Analysis of Algorithms

Last Updated : 13 Jul, 2024

We have discussed Asymptotic Analysis, and Worst, Average, and Best Cases of Algorithms. The main idea of asymptotic analysis is to have a measure of the efficiency of algorithms that don't depend on machine-specific constants and don't require algorithms to be implemented and time taken by programs to be compared. Asymptotic notations are mathematical tools to represent the time complexity of algorithms for asymptotic analysis.

Asymptotic Notations:

- Asymptotic Notations are mathematical tools used to analyze the performance of algorithms by understanding how their efficiency changes as the input size grows.
- These notations provide a concise way to express the behavior of an algorithm's time or space complexity as the input size approaches infinity.
- Rather than comparing algorithms directly, asymptotic analysis focuses on understanding the relative growth rates of algorithms' complexities.
- It enables comparisons of algorithms' efficiency by abstracting away machine-specific constants and implementation details, focusing instead on fundamental trends.
- Asymptotic analysis allows for the comparison of algorithms' space and time complexities by examining their performance characteristics as the input size varies.

- By using asymptotic notations, such as Big O, Big Omega, and Big Theta, we can categorize algorithms based on their worst-case, best-case, or average-case time or space complexities, providing valuable insights into their efficiency.

There are mainly three asymptotic notations:

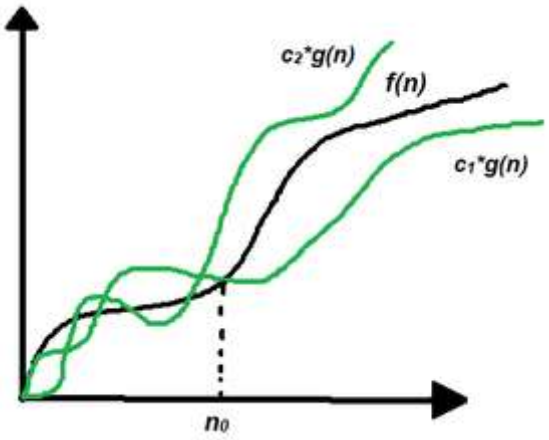
1. ***Big-O Notation (O-notation)***
2. ***Omega Notation (Ω-notation)***
3. ***Theta Notation (Θ-notation)***

1. Theta Notation (Θ-Notation):

Theta notation encloses the function from above and below. Since it represents the upper and the lower bound of the running time of an algorithm, it is used for analyzing the ***average-case*** *complexity of an algorithm.*

.Theta (Average Case) You add the running times for each possible input combination and take the average in the average case.

Let g and f be the function from the set of natural numbers to itself. The function f is said to be Θ(g), if there are constants c1, c2 > 0 and a natural number n0 such that c1* g(n) ≤ f(n) ≤ c2 * g(n) for all n ≥ n0



Theta notation

Mathematical Representation of Theta notation:

> *Θ (g(n)) = {f(n): there exist positive constants c1, c2 and n0 such that 0 ≤ c1 * g(n) ≤ f(n) ≤ c2 * g(n) for all n ≥ n0}*
>
> ***Note:** Θ(g) is a set*

The above expression can be described as if f(n) is theta of g(n), then the value f(n) is always between c1 * g(n) and c2 * g(n) for large values of n (n ≥ n0). The definition of theta also requires that f(n) must be non-negative for values of n greater than n0.

The execution time serves as both a lower and upper bound on the algorithm's time complexity.

It exist as both, most, and least boundaries for a given input value.

A simple way to get the Theta notation of an expression is to drop low-order terms and ignore leading constants. For example, Consider the expression **$3n^3 + 6n^2 + 6000 = \Theta(n^3)$**, the dropping

lower order terms is always fine because there will always be a number(n) after which $\Theta(n^3)$ has higher values than $\Theta(n^2)$ irrespective of the constants involved. For a given function g(n), we denote $\Theta(g(n))$ is following set of functions.

Examples :

{ 100 , log (2000) , 10^4 } belongs to ***Θ(1)***
{ (n/4) , (2n+3) , (n/100 + log(n)) } belongs to ***Θ(n)***
{ (n^2+n) , (2n^2) , (n^2+log(n))} belongs to ***Θ(n^2)***

Note: Θ provides exact bounds.

2. Big-O Notation (O-notation):

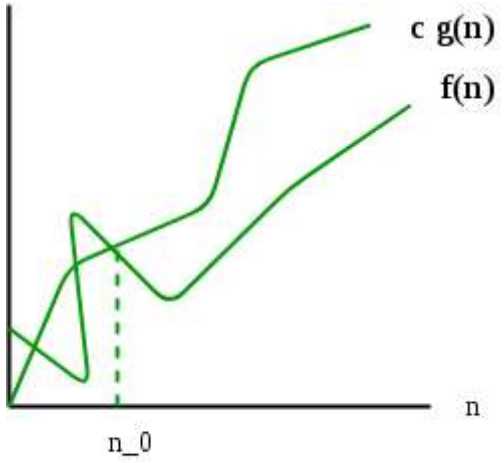
Big-O notation represents the upper bound of the running time of an algorithm. Therefore, it gives the worst-case complexity of an algorithm.

.It is the most widely used notation for Asymptotic analysis.
.It specifies the upper bound of a function.
.The maximum time required by an algorithm or the worst-case time complexity.
.It returns the highest possible output value(big-O) for a given input.
.Big-O(Worst Case) It is defined as the condition that allows an algorithm to complete statement execution in the longest amount of time possible.

If f(n) describes the running time of an algorithm, f(n) is O(g(n)) if there exist a positive constant C and n0 such that, $0 \leq f(n) \leq cg(n)$ for all $n \geq n0$

It returns the highest possible output value (big-O)for a given input.

The execution time serves as an upper bound on the algorithm's time complexity.



f(n) = O(g(n))

Mathematical Representation of Big-O Notation:

O(g(n)) = { f(n): there exist positive constants c and n0 such that 0 ≤ f(n) ≤ cg(n) for all n ≥ n0 }

For example, Consider the case of Insertion Sort. It takes linear time in the best case and quadratic time in the worst case. We can safely say that the time complexity of the Insertion sort is $O(n^2)$.
Note: $O(n^2)$ also covers linear time.

If we use Θ notation to represent the time complexity of Insertion sort, we have to use two statements for best and worst cases:

- The worst-case time complexity of Insertion Sort is $\Theta(n^2)$.
- The best case time complexity of Insertion Sort is $\Theta(n)$.

The Big-O notation is useful when we only have an upper bound on the time complexity of an algorithm. Many times we easily find an upper bound by simply looking at the algorithm.

Examples :

{ 100 , log (2000) , 10^4 } belongs to ***O(1)***
U *{ (n/4) , (2n+3) , (n/100 + log(n)) } belongs to* ***O(n)***
U *{ (n^2+n) , (2n^2) , (n^2+log(n))} belongs to* ***O(n^2)***

Note: *Here,* ***U represents union****, we can write it in these manner because* ***O provides exact or upper bounds .***

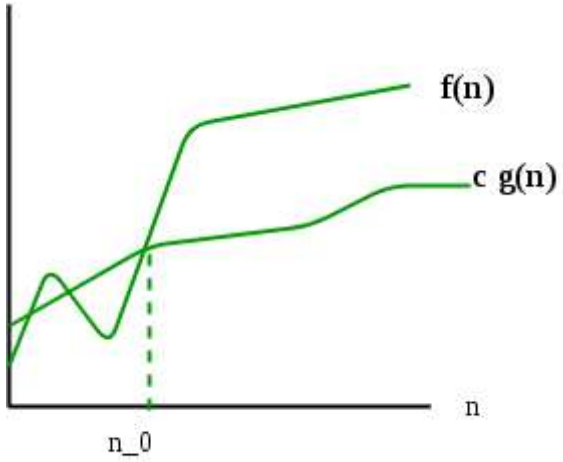
3. Omega Notation (Ω-Notation):

Omega notation represents the lower bound of the running time of an algorithm. Thus, it provides the best case complexity of an algorithm.

The execution time serves as a lower bound on the algorithm's time complexity.

It is defined as the condition that allows an algorithm to complete statement execution in the shortest amount of time.

Let g and f be the function from the set of natural numbers to itself. The function f is said to be Ω(g), if there is a constant c > 0 and a natural number n0 such that c*g(n) ≤ f(n) for all n ≥ n0



f(n) = Omega(g(n))

Mathematical Representation of Omega notation :

Ω(g(n)) = { f(n): there exist positive constants c and n0 such that 0 ≤ cg(n) ≤ f(n) for all n ≥ n0 }

Let us consider the same Insertion sort example here. The time complexity of Insertion Sort can be written as Ω(n), but it is not very useful information about insertion sort, as we are generally interested in worst-case and sometimes in the average case.

Examples :

{ (n^2+n) , (2n^2) , (n^2+log(n))} belongs to ***Ω(n^2)***
U *{ (n/4) , (2n+3) , (n/100 + log(n)) } belongs to* ***Ω(n)***
U *{ 100 , log (2000) , 10^4 } belongs to* ***Ω(1)***

Note: *Here,* ***U represents union,*** *we can write it in these manner because* ***Ω provides exact or lower bounds.***

Properties of Asymptotic Notations:

1. General Properties:

If **f(n)** is **O(g(n))** then **a*f(n)** is also **O(g(n))**, where **a** is a constant.

Example:

f(n) = $2n^2+5$ is $O(n^2)$
*then, 7*f(n) = $7(2n^2+5) = 14n^2+35$ is also $O(n^2)$.*

Similarly, this property satisfies both Θ and Ω notation.

We can say,

*If f(n) is Θ(g(n)) then a*f(n) is also Θ(g(n)), where a is a constant.*
*If f(n) is Ω (g(n)) then a*f(n) is also Ω (g(n)), where a is a constant.*

2. Transitive Properties:

If **f(n)** is **O(g(n))** and **g(n)** is **O(h(n))** then **f(n) = O(h(n))**.

Example:

If f(n) = n, g(n) = n^2 and h(n)=n^3
n is $O(n^2)$ and n^2 is $O(n^3)$ then, n is $O(n^3)$

Similarly, this property satisfies both Θ and Ω notation.

We can say,

If f(n) is Θ(g(n)) and g(n) is Θ(h(n)) then f(n) = Θ(h(n)) .
If f(n) is Ω (g(n)) and g(n) is Ω (h(n)) then f(n) = Ω (h(n))

3. Reflexive Properties:

Reflexive properties are always easy to understand after transitive.

If f(n) is given then f(n) is O(f(n)). Since *MAXIMUM VALUE OF f(n) will be f(n) ITSELF!*

Hence x = f(n) and y = O(f(n) tie themselves in reflexive relation always.

Example:

f(n) = n^2 ; O(n^2) i.e O(f(n))
Similarly, this property satisfies both Θ and Ω notation.

We can say that,

If f(n) is given then f(n) is Θ(f(n)).
If f(n) is given then f(n) is Ω (f(n)).

4. Symmetric Properties:

If **f(n)** is **Θ(g(n))** then **g(n)** is **Θ(f(n))**.

Example:

If(n) = n^2 and g(n) = n^2
then, f(n) = Θ(n^2) and g(n) = Θ(n^2)

This property only satisfies for Θ notation.

5. Transpose Symmetric Properties:

If **f(n)** is **O(g(n))** then **g(n)** is **Ω (f(n))**.

Example:

If(n) = n , g(n) = n^2
then n is O(n^2) and n^2 is Ω (n)

This property only satisfies O and Ω notations.

6. Some More Properties:

1. If **f(n) = O(g(n))** and **f(n) = Ω(g(n))** then **f(n) = Θ(g(n))**
2. If **f(n) = O(g(n))** and **d(n)=O(e(n))** then **f(n) + d(n) = O(max(g(n), e(n)))**

Example:

f(n) = n i.e O(n)
d(n) = n^2 i.e O(n^2)
then f(n) + d(n) = n + n^2 i.e O(n^2)

3. If **f(n)=O(g(n))** and **d(n)=O(e(n))** then **f(n) * d(n) = O(g(n) * e(n))**

Example:

f(n) = n i.e O(n)
d(n) = n^2 i.e O(n^2)
*then f(n) * d(n) = n * n^2 = n^3 i.e O(n^3)*

Note: *If f(n) = O(g(n)) then g(n) = Ω(f(n))*

Important Links :

- There are two more notations called **little o and little omega**. Little o provides a strict upper bound (equality condition is removed from Big O) and little omega provides strict lower bound (equality condition removed from big omega)
- Analysis of Algorithms | Set 4 (Analysis of Loops)
- Recent Articles on analysis of algorithm.

For more details, please refer: Design and Analysis of Algorithms.

Comment | More info | Campus Training Program

Next Article

Getting Started with Array Data Structure

GeeksforGeeks
Sanchhaya Education Private Limited

Corporate & Communications Address:

A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:

K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305





Advertise with us

Company
About Us
Legal
Privacy Policy
Careers
In Media
Contact Us
Corporate Solution
Campus Training Program

Explore
Job-A-Thon
Offline Classroom Program
DSA in JAVA/C++
Master System Design
Master CP
Videos

Tutorials
Python
Java
C++
PHP
GoLang
SQL
R Language
Android

DSA
Data Structures
Algorithms
DSA for Beginners
Basic DSA Problems
DSA Roadmap
DSA Interview Questions
Competitive Programming

Data Science & ML
Data Science With Python
Machine Learning
ML Maths
Data Visualisation
Pandas
NumPy
NLP
Deep Learning

Web Technologies
HTML
CSS
JavaScript
TypeScript
ReactJS
NextJS
NodeJs
Bootstrap
Tailwind CSS

Python Tutorial
Python Examples
Django Tutorial
Python Projects
Python Tkinter
Web Scraping
OpenCV Tutorial
Python Interview Question

Computer Science
GATE CS Notes
Operating Systems
Computer Network
Database Management System
Software Engineering
Digital Logic Design
Engineering Maths

DevOps
Git
AWS
Docker
Kubernetes
Azure
GCP
DevOps Roadmap

System Design
High Level Design
Low Level Design
UML Diagrams
Interview Guide
Design Patterns
OOAD
System Design Bootcamp
Interview Questions

School Subjects
Mathematics
Physics
Chemistry
Biology
Social Science
English Grammar

Databases
SQL
MYSQL
PostgreSQL
PL/SQL
MongoDB

Preparation Corner
Company-Wise
Recruitment Process

More Tutorials
Software Development
Software Testing

Courses
IBM Certification Courses

Programming Languages

Clouds/Devops
DevOps Engineering

GATE 2026
GATE CS Rank Booster
GATE DA Rank Booster

Aptitude Preparation
Puzzles
Company-Wise Preparation
Product Management
Project Management
Linux
Excel
All Cheat Sheets
DSA and Placements
Web Development
Data Science
Programming Languages
DevOps & Cloud
C Programming with Data Structures
C++ Programming Course
Java Programming Course
Python Full Course
AWS Solutions Architect Certification
Salesforce Certified Administrator Course
GATE CS & IT Course - 2026
GATE DA Course 2026
GATE Rank Predictor

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved